



Clock

Updated: 2018-09-06

An abstract base component that provides common timing functionality to all types of clocks. This means all the properties and methods of `Clock` are available on `GlobalClock`, `LocalClock` and `AreaClock`.



You will almost never use the measurements (`time`, `deltaTime`, etc.) provided by clocks directly. You should instead use the same measurements available on the `Timeline` class, since these combine those of all relevant clocks for each `GameObject`.

Properties

```
float localTimeScale { get; set; } = 1
```

The scale at which the time is passing for the clock. This can be used for slow motion, acceleration, pause or even rewind effects.

```
float timeScale { get; }
```

The computed time scale of the clock. This value takes into consideration all of the clock's parameters (parent, paused, etc.) and multiplies their effect accordingly.



`Chronos` is never affected by `Time.timeScale`. You should refrain from using that property, as it may even cause unexpected behaviour with physics. To control time globally, create a root `GlobalClock` instead and make all of your other clocks its descendants.

```
float time { get; }
```

The time in seconds since the creation of the clock, affected by the time scale. Returns the same value if called multiple times in a single frame.



Unlike `Time.time`, this value will not return `Time.fixedTime` when called inside `MonoBehaviour's FixedUpdate`.

```
float unscaledTime { get; }
```

The time in seconds since the creation of the clock regardless of the time scale. Returns the same value if called multiple times in a single frame.

```
float deltaTime { get; }
```

The time in seconds it took to complete the last frame, multiplied by the time scale. Returns the same value if called multiple times in a single frame.



Unlike `Time.deltaTime`, this value will not return `Time.fixedDeltaTime` when called inside `MonoBehaviour's FixedUpdate`.

```
float fixedDeltaTime { get; }
```

The interval in seconds at which physics and other fixed frame rate updates, multiplied by the time scale.

```
float startTime { get; }
```

The unscaled time in seconds between the start of the game and the creation of the clock.

```
bool paused { get; set; }
```

Determines whether the clock is paused. This toggle is especially useful if you want to pause a clock without having to worry about storing its previous time scale to restore it afterwards.

```
GlobalClock parent { get; set; }
```

The parent global clock. The parent clock will multiply its time scale with all of its children, allowing for cascading time effects.

```
ClockBlend parentBlend { get; set; } = Multiplicative
```

Determines how the clock combines its time scale with that of its parent.

Value	Description
-------	-------------

Value	Description
Multiplicative	The clock's time scale is multiplied with that of its parent.
Additive	The clock's time scale is added to that of its parent.



In most cases, multiplicative blend will yield the expected results. However, additive blend becomes extremely useful when you have a parent clock with a time scale of 0.

```
TimeState state { get; }
```

Indicates the state of the clock.

ValueDescriptionAcceleratedTime is accelerated (time scale > 1).NormalTime is in real-time (time scale = 1).SlowedTime is slowed (0 < time scale < 1).PausedTime is paused (time scale = 0).ReversedTime is reversed (time scale < 0).

Methods

```
void LerpTimeScale(float timeScale, float duration, bool steady = false)
```

Changes the local time scale smoothly over the given duration in seconds.



This method is not affected by any time scale.



If you enable the steady parameter, duration will apply to a time scale variation of 1.

For example, if you call `LerpTimeScale(3, 2, true)`, and the current time scale is 1, the duration of the lerp will be $(3 - 1) * 2 = 4$ seconds instead of 2.

```
void ComputeTimeScale()
```

The time scale is only computed once per update to improve performance. If you need to ensure it is correct in the same frame, call this method to compute it manually.

Examples

Toggle a pause on the "World" clock when pressing P:

```
using UnityEngine;
using Chronos;

class MyBehaviour : MonoBehaviour
{
    void Update()
    {
        if (Input.GetKeyDown(KeyCode.P))
        {
            GlobalClock worldClock = Timekeeper.instance.Clock("World");

            worldClock.paused = !worldClock.paused;
        }
    }
}
```

Same example, this time with smoothing:

```
using UnityEngine;
using Chronos;

class MyBehaviour : MonoBehaviour
{
    bool paused = false;

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.P))
        {
            GlobalClock worldClock = Timekeeper.instance.Clock("World");

            if (!paused)
            {
                worldClock.LerpTimeScale(0, 1); // Change time scale to 0 over 1 second
                paused = true;
            }
            else
            {
                worldClock.LerpTimeScale(1, 1); // Change time scale to 1 over 1 second
                paused = false;
            }
        }
    }
}
```

[Previous](#)

← **Timekeeper**

A global singleton tasked with keeping track of global clocks in the scene.

[Next](#)

A Clock that affects all Timeline and other global clocks configured as its children.

Was this article helpful?

Be the first to vote!

👍 Yes, helpful

👎 No, not for me



PRODUCTS

LUDIQ

Contact

Blog